# 4. Files and troubleshooting

## 4.1 Where everything is kept

| What | Where |
|---|---|
| Installed programs | `/usr/local/bin/treesync-root-gcc`, `treesync-home-gcc`, `treesync-cli-gcc`, the same with `-clang`, and `treesync-cli` (a link) (`root:root`, 755; by TreeSyncInstall / `install.sh`) |
| Installed manual | `/usr/local/share/doc/treesync/pdf/`, and `INSTALLED-gcc`, `INSTALLED-clang`: which build, commit, when |
| App entries, installed | `/usr/local/share/applications/treesync-root-gcc.desktop`, `treesync-home-gcc.desktop`, and the same with `-clang` |
| polkit action | `/usr/share/polkit-1/actions/org.treesync.run-as-root.policy` |
| Built programs | `v001/out/gcc/` and `v001/out/clang/`: `treesync-root`, `treesync-home`, `treesync-cli`, `treesync-build`, `treesync-install`, `treesync-test` |
| Build and test output | `v001/out/` (`verify/` logs, `bootstrap-*.txt`); ignored by git |
| GUI settings | `~/.config/treesync/settings` (TreeSyncRoot), `settings-home` (TreeSyncHome): one option per line, as on the command line |
| TreeSyncBuild's choices (and the display) | `~/.config/treesync/build` |
| TreeSyncBuild and TreeSyncInstall in your app list | `~/.local/share/applications/treesync-build.desktop`, `treesync-install.desktop` (by `setup/app-folder.sh`) |
| The TreeSyncApp folder | GNOME: `gsettings get org.gnome.desktop.app-folders folder-children`; other desktops: `~/.config/menus/applications-merged/treesyncapp.menu` |
| Database | `~/.local/share/treesync/treesync.db` (SQLite): runs, actions, hash cache, checks |
| Run logs | `~/.local/share/treesync/logs/YYYY-MM-DD_HHMMSS.log` |
| Results of every run | `~/.local/share/treesync/logs/Results.txt`, one line per run |
| AI exports | `~/.local/share/treesync/logs/ai/run-N-logs.AI.md`, `run-N-sql.AI.md` |
| Root runs | the same under `/root/.local/share/treesync/` |
| Safe-delete folder | `<target>.old` next to the target; `<target>/.treesync-old` inside it when the target is the top folder of a disk (or `--safe-delete-dir`) |
| Unfinished copies | `NAME.treesync-part` next to where the file goes; removed when done |

| What | Where |
|---|---|
| This manual | `Manual/*.md` (the sources) and `Manual/pdf/` (built by `Manual/build-pdf.sh`) |

`--db=PATH` puts the database (and its `logs/` folder) somewhere else.

## 4.2 Troubleshooting

**"... is outside your home directory. Without root, TreeSync only works inside ..."**
That is TreeSyncHome (or `treesync-cli` running as you). To sync other folders use **TreeSyncRoot**, which syncs as root for administrators. Inside your home nothing is needed.

**"TreeSyncRoot is for administrators"**
You are not an administrator, so you sync with TreeSyncHome, as yourself, inside your home. To sync other folders, an administrator does it, or adds you to wheel (`sudo usermod -aG wheel NAME`; log in again), which also lets you use sudo.

**TreeSyncRoot and TreeSyncHome are not in the app list, or not in the TreeSyncApp folder**
They are installed by TreeSyncInstall; until then only TreeSyncBuild and TreeSyncInstall are there. The folder is per user: run `v001/setup/app-folder.sh` as yourself (TreeSyncInstall and bootstrap do). GNOME may need you to log out and in once to show a new folder.

**Root runs use an older TreeSync, or the log says the installed one "is not this build"**
The installed copy does not follow `out/`: install again with TreeSyncInstall (or `./install.sh`) after building. TreeSyncInstall shows which build is installed, and so does `cat /usr/local/share/doc/treesync/INSTALLED-gcc` (or `-clang`).

**The password is asked at every run**
TreeSyncRoot asks once per window; if each new window asks at once, the polkit action is missing: it is installed only when `install.sh` runs as root (the default, through sudo), not for a `--prefix` of your own.

**`sudo treesync-cli`: "command not found"**
sudo does not search `/usr/local/bin` on RHEL. Leave the sudo out: the installed `treesync-cli` starts itself through sudo.

**TreeSync is not in the app list**
Run `./install.sh`; log out and in again if the desktop does not pick it up at once.

**A path is refused**
Without root, a path that leaves your home (also through `..` or a symlink) is refused. Use root, or a path inside your home.

**bootstrap stops: "Still missing: ..."**
A required package did not install. Run the install command it printed by hand and read its error (often a repository that is not enabled, for example CodeReady Builder on RHEL for `gtk3-devel`), then run `./bootstrap.sh` again.

**build.sh: "U++ (umk + uppsrc) not found"**
Run `./bootstrap.sh`, or point to an existing U++ with `export UPP_DIR=/path/to/u++`.

**gcc build fails to link: "undefined reference to `handle_contract_violation`"**

GCC 16 (Fedora 44) has native C++26 contracts and keeps their default violation handler in `libstdc++exp`, which is linked only with `-fcontracts`. `build.sh` adds it for GCC 16 and newer (not for GCC 15, where `-fcontracts` means something else). An older checkout, or a build method of your own, does not: update TreeSync, or add `-fcontracts` to the build method's link options.

**Building as root fails: "U++ not found", or umk "error while loading shared libraries"**

Older scripts looked for U++ in root's home and then picked a U++ built for another distribution. Current ones use the U++ of the folder's owner (`~owner/u++`) and skip one that cannot run here. Root-owned leftovers in `out/` from such a build: `sudo chown -R $USER v001/out`.

**A gcc release build crashes at start**

The U++ heap fix is missing (`uppfix`); `./bootstrap.sh` applies it.

**A test fails**

`./verify.sh` stops and prints the failing checks; the full output is in `v001/out/verify/<compiler>-<type>.log`. `git push --no-verify` pushes anyway, but fix it first if you can.

**A run is slow**

Make sure the GUI runs a **release** build (`./verify.sh` leaves them in `out/`). Many small files: more threads (`--threads=8`) can help. Over SFTP one stream is often slow while several together are not (each waits for its writes and its checks); with *Split big files across idle threads* on (normal) every thread works on the last big files, for a local source. `--compress` helps text, not media.

**"no Wayland session found" in TreeSyncBuild**

The GUI was started without `WAYLAND_DISPLAY`, usually through `sudo`. Start it as yourself, and use TreeSyncRoot for root runs.

**The host key is refused (remote trees)**

Connect once with `ssh user@host` to check and accept the key; TreeSync never accepts an unknown or changed key by itself.

**"the copies may not fit" (exit code 4)**

The room check found that the new and changed files, at full size, are more than the target disk has free. Make room (**Clear .old** frees what earlier runs moved aside), or start anyway: files with runs of zeros (VM images) need much less than their size, and the run stops by itself before the disk is full.

**"stopping: the target disk is nearly full"**

The run stopped itself to keep a reserve free (see 3.5). Nothing is lost: make room, then **Run** again and it continues where it stopped.

**A right-click change says "Not changed"**

The message says why: a name that already exists (Rename never replaces), a name TreeSync keeps for itself, or no permission (TreeSyncHome changes only what you may change; TreeSyncRoot changes anything).

**The copy is smaller than the source in `du`, or in the disk's free space**

Runs of zeros (common in VM images) are kept as holes, so the copy takes less disk than its size. `ls -l` shows the same size on both sides, and so does the check at the end of the run, with the same SHA-256 on both sides. The check says so in its "on disk" line.

**A run says it copied fewer bytes than the file has**

It resumed a stopped run. The bytes the stopped run had copied were read back, checked and kept, so only the rest was copied. The check at the end says how much came from the stopped run, and shows both sides' full size and SHA-256.

**A run was stopped and the target has `.treesync-part` files**

Run the same sync again: it resumes, re-checks what was already copied, and removes the part files when done. `--fresh` starts over instead.

**"cannot move ... it is on a different disk"**

A move cannot cross disks. A target at the top of a disk gets its safe-delete folder inside it (`.treesync-old`) by itself; this message means `--safe-delete-dir` points to another disk. Use a folder on the target's disk, or no `--safe-delete-dir`.

**The `.old` folder keeps growing**

Safe delete never deletes. **Sync > Clear .old folders...** (or `treesync-cli --clear-old --no-dry-run`) empties it when you are sure.

**Something else**

**View > Export newest run for AI** and give the files in `logs/ai/` to an AI, or read them yourself: they list every problem of the run first.